

ENTERPRISE PLATFORM

High-Level Design Document

HLD — PART 2 OF 3

Component Design | Data Architecture | Multi-Tenancy | Security | Event-Driven Messaging

Document Scope

This is Part 2 of the 3-part HLD. Part 1 covered the executive summary, architecture decision, system topology, module map, deployment phases, API strategy, and non-functional requirements. This part covers the internal design of every major component, data architecture, multi-tenancy strategy, security architecture, and the event-driven messaging system.

9. Component Design — Every Major Component

Each component in the system has a single, well-defined responsibility. This section details what each component does, what it does not do, and how it connects to others.

9.1 API Gateway Layer

The API Gateway is the single entry point for all client requests. No client ever communicates directly with a backend service. In Phase 1 this is Nginx. In Phase 3 this becomes Kong or AWS API Gateway.

Responsibility	Implementation
TLS Termination	Nginx handles HTTPS. Backend services communicate over HTTP internally — no TLS overhead between services
Request Routing	/api/* routes to FastAPI backend. /* routes to Next.js frontend. /metrics restricted to internal IPs
Rate Limiting — Nginx Level	OTP: 5 req/min. Login: 10 req/min. Global: 200 req/min per IP. First line of defence before requests reach application
Rate Limiting — Application Level	Redis sliding window inside FastAPI. Second line of defence with precise per-endpoint control
Request ID Injection	Every request receives a unique UUID. Logged at Nginx level and propagated to all downstream services
Security Headers	X-Frame-Options, HSTS, CSP, X-Content-Type-Options applied at gateway level
Health Check Routing	/health and /readyz routed to backend. Kubernetes liveness and readiness probes use these
Static Asset Caching	Next.js /_next/static/* cached for 1 year with immutable cache headers

Phase 3 migration path: Nginx config is replaced by Kong declarative config. The upstream services (FastAPI pods) remain unchanged. The routing rules translate directly.

9.2 Authentication and Identity Service (Module 1)

The authentication service is the most critical component in the platform. Every other module depends on it. It is therefore designed with the strictest internal boundaries and the most conservative change policy.

Sub-Component	Responsibility
Registration Handler	Validates input, checks uniqueness, creates user record, dispatches email

Sub-Component	Responsibility
	and SMS verification OTPs
OTP Engine	Generates HMAC-signed OTPs, stores SHA-256 hash in Redis with TTL, validates with constant-time comparison, enforces one-time use
Token Issuer	Creates JWT access tokens (15 min) and refresh tokens (7 days), stores refresh token hash in PostgreSQL session table
Session Manager	Validates refresh tokens, detects reuse attacks, rotates token pairs, maintains session audit trail
OAuth Handler	Manages Google OAuth PKCE flow, CSRF state validation, account linking for existing users
Password Reset	Cryptographic reset token generation, 30-minute expiry, all-session invalidation on reset
Rate Limit Enforcer	Redis sliding-window per endpoint per IP. Integrates with token issuer to detect credential stuffing
Audit Logger	Immutable append-only event log. Every security event recorded with IP, user agent, timestamp, outcome

Module 1 Contract — What Other Modules Must Never Do

No other module may read or write the users table directly. No other module may issue or validate JWT tokens directly. All identity operations go through Module 1's public interface. This boundary is enforced by code review policy and, in Phase 3, by network policy between microservices.

9.3 Frontend Application (Next.js)

Rendering Strategy	When It Is Used
Server-Side Rendering (SSR)	Protected pages — dashboard, profile, orders. Server validates cookie and fetches user data before sending HTML. No flash of unauthenticated content.
Static Generation (SSG)	Public pages — landing page, pricing, documentation. Built at deploy time. Served from CDN edge. Zero server load.
Client-Side Rendering (CSR)	Interactive widgets — real-time notifications, live dashboards. Data fetched after initial page load.
Edge Middleware	Auth guard runs at CDN edge. Unauthenticated requests to protected routes are redirected before hitting the origin server.

9.4 Notification Service (Module 4 — Planned)

The notification service is treated as a first-class component even though it is not yet built, because Module 1 already sends emails and SMS. The current implementation in `notification_service.py` will be extracted into Module 4 without changes to callers.

Current (Module 1 inline):

`auth.py` → `notification_service.py` → SendGrid/Twilio

Future (Module 4 extracted):

`auth.py` → `events.publish('otp.requested', {...})`
→ Module 4 subscribes → SendGrid/Twilio

The auth module emits an event.

It does not care how the notification is delivered.

Module 4 handles all delivery logic.

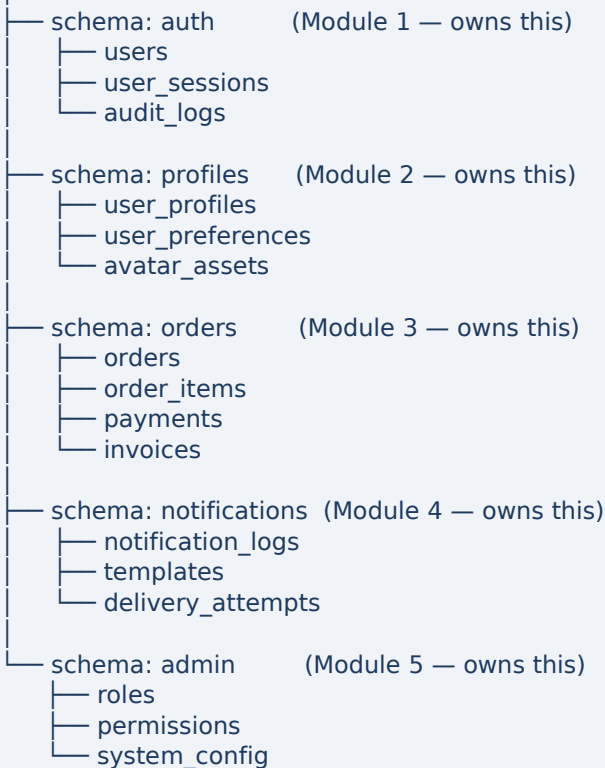
This decoupling means delivery providers can change without touching the auth module at all.

10. Data Architecture

10.1 Database Per Module Schema Strategy

All modules share one PostgreSQL instance but each module owns its own schema. A schema in PostgreSQL is a namespace — like a folder inside the database. This gives logical isolation without the cost of separate databases.

PostgreSQL Database: enterprise_platform



Cross-schema references use database foreign keys sparingly. The `auth.users.id` is the universal primary key referenced by all other schemas. All other cross-module data access goes through the Module 1 API — never direct database joins across schemas.

10.2 Data Ownership Rules

Rule	Description	Enforcement
Single Owner	Each table is owned by exactly one module. No module writes to another module's tables.	Code review policy. Database user permissions per schema in Phase 3.
API Access Only	Cross-module data access goes	Architecture review. Banned in CI

Rule	Description	Enforcement
	through the owning module's API, never direct SQL.	linting rules.
Event for Async	Modules that need to react to another module's changes subscribe to events, never poll the database.	Event bus architecture. No scheduled cross-schema queries.
Read Models	Modules can maintain their own read-optimised copies of data they frequently need.	Populated via events. Module 3 keeps a local user snapshot for order history.

10.3 Redis Data Architecture

Redis stores only ephemeral data — data that can be regenerated or re-fetched if lost. It is never the system of record for anything.

Key Pattern	Data Stored	TTL
otp:{channel}:{identifier}	OTP hash + HMAC signature + attempt count	10 minutes
pwd_reset:{token_hash}	User ID for password reset	30 minutes
rl:{endpoint}:{ip}	Sorted set of request timestamps	Window duration (900s)
session_cache:{user_id}	Serialised user object for fast /me lookups	15 minutes
events:{stream_name}	Module event stream (Redis Streams)	7 days retention
lock:{resource}:{id}	Distributed lock for critical sections	30 seconds

11. Multi-Tenancy Architecture

As an IT services company, this platform will potentially serve multiple clients (tenants) from a single deployment. Multi-tenancy must be designed from Module 1 even if not activated immediately.

11.1 Three Multi-Tenancy Models Evaluated

Model	Isolation	Cost	Decision
Database per tenant	Highest	Highest — one DB instance per client	Rejected — too expensive at low client count
Schema per tenant	High	Medium — one schema per client in shared DB	Chosen for Phase 2 when multi-tenancy is activated
Row-level tenancy	Lowest	Lowest — tenant_id column on every table	Used for Phase 1 — simple, zero extra cost

11.2 Phase 1 — Row-Level Tenancy (Current Implementation)

Every table that needs tenant isolation has a tenant_id column. A default tenant (the platform itself) is used until the multi-tenant feature is activated.

```
-- Every table that needs tenant isolation:
ALTER TABLE auth.users
  ADD COLUMN tenant_id UUID NOT NULL
  DEFAULT '00000000-0000-0000-0000-000000000001';

-- Row-level security policy:
CREATE POLICY tenant_isolation ON auth.users
  USING (tenant_id = current_setting('app.tenant_id')::uuid);

-- Application sets this at the start of each request:
await db.execute(text(
  "SET LOCAL app.tenant_id = :tid",
  {'tid': str(request.state.tenant_id)}
))

-- Now all queries automatically filter by tenant
-- No developer can accidentally leak cross-tenant data
```

11.3 Tenant Resolution Strategy

The system identifies which tenant a request belongs to through one of three mechanisms:

Method	Example	When Used
Subdomain	client1.yourplatform.com	Multi-tenant SaaS deployment — each client has a subdomain
Custom Domain	client1.com → your platform	White-label deployment for enterprise clients
JWT Claim	tenant_id embedded in access token	API access — tenant resolved from authenticated user's token

12. Security Architecture

Security is not a feature that is added later. Every architectural decision below was made with a specific threat in mind. This section documents the threat, the architectural response, and where in the code it is implemented.

12.1 Authentication Security Model

Threat	Attack Description	Architectural Defence
Token theft via XSS	Malicious JS reads token from localStorage and exfiltrates it	All tokens in HttpOnly cookies — JS cannot access them under any circumstance
CSRF attack	Malicious site submits form using victim's cookies	SameSite=Strict cookies + CORS allowlist. Cross-origin requests cannot carry cookies.
Session hijacking	Attacker intercepts refresh token and uses it indefinitely	Refresh token rotation — every use issues a new token and invalidates the old one
Token reuse attack	Attacker uses stolen refresh token after victim has rotated it	Reuse detection — if an invalidated token is presented, ALL sessions are immediately killed
Credential stuffing	Bot tries millions of email/password combinations from breached databases	Redis rate limiting 10 attempts per 10 minutes per IP + bcrypt cost factor 12 slows every attempt
OTP brute force	Bot tries all 1,000,000 possible 6-digit codes	5 wrong attempts invalidate the OTP. Rate limit 5 OTPs per 15 minutes per IP.
OTP replay	Attacker intercepts OTP and reuses it	OTP deleted from Redis on first successful use. One-time use enforced at data layer.
Email enumeration	Attacker discovers which emails are registered	All OTP request endpoints return 202 regardless of whether email/phone exists
SQL injection	Attacker injects SQL via input fields	SQLAlchemy ORM parameterises all queries. String concatenation in SQL is architecturally banned.
Timing attack on passwords	Attacker measures response time to determine if email exists	bcrypt always runs even for unknown users. Constant-time comparison in all crypto operations.

12.2 Authorisation Model — RBAC

Role-Based Access Control is designed here and will be implemented in Module 5 (Admin). The architecture is defined now so Modules 2, 3, and 4 can reference it consistently.

Role hierarchy:

SUPER_ADMIN

- Full system access. Platform operator only.

TENANT_ADMIN

- Full access within their tenant.
- Cannot access other tenants.
- Cannot modify platform configuration.

MANAGER

- Read/write access to their tenant's business data.
- Cannot manage users or system config.

USER (default)

- Access to their own data only.
- Standard application user.

READONLY

- Read-only access to permitted resources.
- Used for auditors, support staff.

Permission check in any endpoint:

```
@require_permission('orders:write')
async def create_order(...):
```

```
...
```

```
# Permission format: {module}:{action}
```

```
# Examples:
```

```
# auth:read_any    — read any user (admin only)
```

```
# profiles:write   — update profile
```

```
# orders:write     — create/update orders
```

```
# admin:dashboard  — view admin dashboard
```

12.3 Secrets Management Architecture

Phase	Secrets Store	How Secrets Are Accessed
Phase 1	Local .env file	Pydantic Settings reads at startup. File has 600 permissions. Never in Git.
Phase 2	AWS Secrets Manager	ECS task role reads secrets at container startup. Developers never see production values.
Phase 3	HashiCorp Vault	Kubernetes Vault Agent injects secrets as environment variables. Zero human access to production secrets.

13. Event-Driven Messaging Architecture

Events are the mechanism through which modules communicate asynchronously. A module publishes an event when something significant happens. Other modules subscribe to events they care about. The publisher never knows who the subscribers are.

Why Events Are Critical for This Architecture

Without events, adding Module 2 would require modifying Module 1's code to call Module 2's profile creation function. That creates coupling — Module 1 would need to know about Module 2. With events, Module 1 publishes 'user.registered' and Module 2 subscribes to it. Module 1 never changes when new modules are added. This is the foundation of the pluggable module architecture.

13.1 Event Catalogue — All Planned Events

	Event Name	Published By	Subscribed By
1	auth.user.registered	Module 1 — Auth	Module 2 (create profile), Module 4 (send welcome email)
2	auth.user.verified	Module 1 — Auth	Module 4 (send verified confirmation)
3	auth.login.success	Module 1 — Auth	Module 5 (admin audit feed), Module 4 (login alert if new device)
4	auth.login.failed	Module 1 — Auth	Module 5 (security alerts), Module 4 (breach notification if threshold)
5	auth.password.reset	Module 1 — Auth	Module 4 (send confirmation email), Module 5 (security log)
6	profiles.updated	Module 2 — Profiles	Module 3 (update order contact info), Module 5 (audit)
7	orders.created	Module 3 — Orders	Module 4 (send order confirmation), Module 5 (revenue report)
8	orders.paid	Module 3 — Orders	Module 4 (send receipt), Module 5 (revenue dashboard)
9	orders.cancelled	Module 3 — Orders	Module 4 (send cancellation), Module 5 (report)
10	notifications.failed	Module 4 — Notifications	Module 5 (alert if delivery failure rate exceeds threshold)

13.2 Event Schema Standard

Every event published on the bus must follow this schema. Consistency allows any module to process any event without custom parsing logic.

```
{
  "event_id": "uuid-v4",          // unique per event
  "event_type": "auth.user.registered", // dot-namespaced
  "version": "1.0",              // event schema version
  "timestamp": "2024-01-16T14:32:01Z", // UTC ISO 8601
  "tenant_id": "uuid",           // for multi-tenancy
  "actor_id": "uuid | null",     // user who caused event
  "data": {
    // event-specific payload
    "user_id": "uuid",
    "email": "user@example.com",
    "method": "email_otp"
  },
  "metadata": {
    "source": "auth-service", // which service emitted
    "request_id": "uuid",     // for distributed tracing
    "ip_address": "103.x.x.x"
  }
}
```

13.3 Event Bus Implementation by Phase

Phase	Technology	Reason
Phase 1	Redis Streams	Already in the stack. Zero extra cost. Handles 10K users easily. Consumer groups for competing consumers.
Phase 2	Redis Streams or AWS SQS/SNS	SQS/SNS when managed reliability is needed. Fan-out via SNS to multiple SQS queues per module.
Phase 3	Apache Kafka	Handles millions of events per second. Event replay for new modules to catch up on history. Exactly-once delivery.

The event publisher interface is abstracted behind app/events/bus.py. When upgrading from Redis Streams to Kafka, only the bus.py implementation changes. Every module that calls events.publish() continues working without modification.

14. Caching Architecture

Caching reduces database load and improves response times. The caching strategy is layered — each layer serves a different purpose.

Layer	Technology	TTL	What Is Cached
CDN Edge	Cloudflare	1 year	Static assets — JS, CSS, images, fonts
CDN Edge	Cloudflare	10 minutes	Public SSG pages — landing page, pricing
Application	Redis	15 minutes	User session data for /me endpoint
Application	Redis	5 minutes	Permission/role lookups for RBAC checks
Application	Redis	1 hour	Configuration values from system_config table
Database	PostgreSQL	Always	Frequently-read tables via pg_prewarm on startup
Browser	HTTP headers	15 min	API responses for non-sensitive public data

14.1 Cache Invalidation Strategy

- User session cache — invalidated immediately on logout, password reset, or role change
- Permission cache — invalidated when admin changes a user's role in Module 5
- Config cache — invalidated when system_config is updated via admin API
- Pattern: write-through for critical data, TTL expiry for read-heavy non-critical data
- Never cache: OTPs after verification, reset tokens after use, financial transaction data

15. Observability Architecture

Observability means knowing what the system is doing at all times — not just when it breaks. The three pillars of observability are logs, metrics, and traces.

Pillar	Phase 1	Phase 2	Phase 3
Logs	structlog JSON to stdout. Docker captures. Manual review.	CloudWatch Logs. Log groups per service. Search and filter.	Loki + Grafana. Centralised log aggregation. Alert on

Pillar	Phase 1	Phase 2	Phase 3
			patterns.
Metrics	Prometheus /metrics endpoint. Manual check.	CloudWatch Metrics. Dashboards. Alarm on thresholds.	Prometheus + Grafana. Custom dashboards per module. PagerDuty alerts.
Traces	Request ID in all logs. Manual correlation.	AWS X-Ray. End-to-end request tracing.	Jaeger. Distributed tracing across all microservices.
Errors	Sentry free tier. Email on first occurrence.	Sentry Team. Grouped errors. Assignment workflow.	Sentry Business. Performance monitoring. Session replay.
Uptime	UptimeRobot free. Email on downtime.	AWS CloudWatch Synthetics. SLA reporting.	Multi-region synthetic monitoring. SLA dashboard.

HLD Part 2 Complete

Part 3 covers: Scalability and performance strategy, cost model per phase, integration contracts for each future module, the full evolution roadmap from Modular Monolith to Microservices, and the team/process recommendations. Reply 'HLD Part 3' to continue.